IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 30 January 2026, accepted 16 February 2026, date of publication 18 February 2026, date of current version 23 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3665975

RESEARCH ARTICLE

Improvement of Drone Gimbal System Performance Using a Parallel Structure of Explicit Model Predictive Control and Adaptive Neuro-Fuzzy Inference System

KANG-HYEON CHOI AND HYUK-JUN CHANG, (Member, IEEE)
School of Electrical Engineering, Kookmin University, Seoul 02707, Republic of Korea

Corresponding author: Hyuk-Jun Chang (hchang@kookmin.ac.kr)

This work was supported by Korea Research Institute for Defense Technology Planning and Advancement (KRIT)–Grant funded by Defense Acquisition Program Administration (DAPA) under Grant KRIT-CT-23-041. Also, Following are results of a study on the "Convergence and Open Sharing System" Project, supported by the Ministry of Education and National Research Foundation of Korea.

ABSTRACT Gimbal control is a crucial technology for drones that ensures camera stabilization and precise control of shooting angles, thereby enhancing the performance of autonomous aerial applications. The performance of the gimbal system is influenced by the motor inertia and external disturbances. In this study, the control performance of the gimbal system is improved using explicit model predictive control (EMPC), with external disturbances compensated by an adaptive neuro-fuzzy inference system (ANFIS). EMPC divides the control domain into polyhedral regions and calculates the control input by applying an optimal control law within each region. ANFIS is a control technique that integrates fuzzy logic with artificial neural networks. ANFIS learns the relationship between the inputs and outputs from the training data to construct a rule-based system. This study proposes a controller design methodology that connects EMPC and ANFIS in a parallel structure. ANFIS estimates and compensates for disturbances at each sampling interval. Since EMPC utilizes precomputed control laws based on critical regions, the parallel configuration with ANFIS reduces the computational complexity. Additionally, EMPC predicts the system's behavior using a plant model and applies constraints, ensuring that the overall system remains stable and operates within a predictable range, even as ANFIS compensates for disturbances. To the best knowledge of the authors, this study presents the first experimental application of a parallel EMPC–ANFIS structure on a QUBE-Servo 3 drone gimbal system.

INDEX TERMS Drone gimbal system, model predictive control (MPC), explicit model predictive control (EMPC), adaptive neuro-fuzzy inference system (ANFIS).

I. INTRODUCTION

Gimbal control for drones is a fundamental technology that ensures camera stability during flight and enables precise control of the shooting angle in the desired direction [1]. Initially introduced in the aviation and maritime sectors, gimbal control for autonomous aerial vehicles was used to collect real-time information for surveillance and reconnaissance [2]. With the advancement in electronic control and motor drive technologies, it has since been miniaturized and improved in precision, leading to its widespread application in the drone industry [3], [4], [5]. Drones are continuously subjected to rapid attitude changes and external disturbances, making the stability and responsiveness of the gimbal system essential for precise image acquisition, target tracking, and measurement accuracy [6], [7]. Today, gimbal control is used not only in aerial photography drones but also in various fields, such as disaster monitoring, military reconnaissance, and real-time remote sensing [8], [9]. In these applications, the precision and stability of the gimbal control system

The associate editor coordinating the review of this manuscript and approving it for publication was Wanqing Zhao.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

directly impact the efficiency and reliability of drone-based operations. As a result, numerous studies aimed at improving the stability and accuracy of drone gimbal control systems are critical for enhancing the performance of high-precision autonomous aerial applications [10], [11], [12].

Ahmad proposes a method that enhances the stability of a 2-degree-of-freedom (DOF) gimbal system using a PID controller [12]. He designs the PID controller by determining the proportional, integral, and derivative gains through a trial-and-error tuning method. As a result, the proportional gain reduces the rise time, the integral gain eliminates the steady-state error, and the derivative gain reduces the overshoot, thereby improving the stability and performance of the gimbal system. The PID controller is generally effective in stably controlling the vibrations of various frequency components in 2-DOF motion. However, its performance depends on the specific system. When the structure of the drone gimbal system or certain system components are modified, a PID controller designed for the previous system may suffer performance degradation, necessitating parameter retuning.

Abdo proposes a method that improves the performance of a two-axis gimbal system by compensating for the disturbance torque using a fuzzy PID-type controller [11]. The fuzzy controller enhances the system stability and performance by generating appropriate outputs based on a predesigned rule base in response to current inputs. The fuzzy PID-type controller can maintain stable system performance by compensating for disturbances caused by external factors. However, it has the limitation that it cannot ensure stable performance when the input variables exceed the range defined by the rule base. If torque disturbances from external factors (e.g., strong wind or voltage disturbances from internal faults) occur, the input variables of the fuzzy controller may exceed the predefined rule base. In this case, the drone gimbal system can no longer guarantee stable performance.

This study proposes a methodology to improve gimbal control performance using explicit model predictive control (EMPC) and to compensate for disturbances caused by external factors through an adaptive neuro-fuzzy inference system (ANFIS). Model predictive control (MPC) is a technique that predicts future behavior based on the system's dynamic model and calculates the control input [13], [14]. MPC predicts the system's output over a specific time horizon based on its current state [15]. It then solves an optimization problem to determine the control input that makes the predicted output match the reference input as closely as possible, obtains the control input, and applies only the first input value [16]. The process is repeated in the next sampling period, with the system's new state measured [17]. During the MPC design process, constraints on the state, input, and output variables can be incorporated [18]. Additionally, since MPC calculates the control input by considering the impact of the current input on the system's future behavior, it can maintain a robust control performance [19], [20]. However, MPC requires solving an optimization problem at each sampling period, making it computationally intensive and unsuitable for high-speed real-time control [21]. Moreover, the computation time may vary, and if real-time optimization fails, the control performance of MPC may not be guaranteed [22]. In summary, while MPC is a powerful technique for predicting future states and computing control inputs, it faces limitations in real-time control applications.

EMPC determines control inputs quickly by utilizing precomputed optimal solutions, rather than solving the optimization problem in real time, as in conventional MPC [23]. While MPC solves an optimization problem at each sampling period, EMPC analyzes the entire state space offline and stores the solutions in advance [24]. In other words, EMPC determines the control input by referencing precomputed polynomials based on the system's state. EMPC operates by partitioning the control domain into polyhedral regions and precomputing the optimal control law for each region in advance [25]. Since EMPC can quickly find the control input using a lookup table without solving an optimization problem at each step, EMPC has a lower computational burden than conventional MPC and eliminates computation delays, thereby improving system stability [26]. Additionally, while maintaining the constraint-handling capability of MPC, EMPC enables real-time implementation, making it ideal for applications where high speed is essential, such as robotics and drones [27], [28].

ANFIS is a technique that combines fuzzy logic control and artificial neural networks, as proposed by Jang [29], [30], [31]. Fuzzy logic control applies fuzzy logic to control systems, introduced by Zadeh [32], [33]. Fuzzy logic control sets input and output variables and designs rule-based systems through the fuzzification process [34]. Then, through the fuzzy inference process, the weights of each rule are calculated to determine the overall output, and the defuzzification process is used to convert the output into a usable crisp value [35]. Through this intuitive control approach, fuzzy logic control provides robust performance even in complex situations [36], [37], [38]. Artificial neural networks are algorithms that mimic the structure of neurons in the human brain to learn and process information. Multiple nodes are hierarchically connected, receiving input data, applying weights, and repeatedly finding the output through an activation function [39]. Neural networks can automatically learn patterns from data [40]. Without the need for a person to manually define rules, the neural network can find the rules by providing training data. A trained neural networks can generate appropriate outputs for new input data as well [41]. In other words, neural networks have the ability to solve similar problems beyond the learned data. ANFIS leverages both the interpretability of fuzzy logic and the learning ability of neural networks. ANFIS maintains the structure of fuzzy rules while using neural networks to automatically generate and adjust rules through data-driven

learning [42], [43]. Additionally, ANFIS has a fast computation speed, making it suitable for real-time system control. In summary, ANFIS combines the intuitiveness of the fuzzy control with the automatic learning ability of neural networks, making it a powerful control technique for efficiently modeling and optimizing nonlinear systems. It can adjust the rules based on real-world data, providing higher control performance and adaptability.

This study presents a control methodology that integrates EMPC and ANFIS to enhance the performance and robustness of the gimbal system. The gimbal's performance is mainly evaluated based on its transient response characteristics. To achieve rapid and stable real-time operation, it is crucial to minimize the rise time and reduce the effects of inertia on the motor angle control by limiting the settling time and overshoot. The proposed EMPC approach predicts the future behavior of the system and computes the optimal control input accordingly. Unlike conventional PID controllers, which rely on fixed parameters and exhibit significant sensitivity to time delays, EMPC offers superior performance by employing a system model to obtain optimal solutions. Moreover, EMPC improves the real-time computational efficiency by partitioning the state space into polyhedral regions in advance and selecting the appropriate control input based on this partitioning, making it particularly suitable for applications that require fast response, such as gimbal control.

In drone systems, the performance can be significantly affected by mechanical imperfections and environmental disturbances. Internal faults may lead to voltage disturbances, whereas external factors like strong winds or physical impacts can introduce torque disturbances, both of which can degrade the control performance. To address these issues, ANFIS is employed to compensate for such disturbances and improve system stability. By learning the nonlinear mapping between inputs and outputs through a neural network structure, ANFIS generates control inputs tailored to specific conditions. This approach overcomes the limitations of conventional fuzzy logic controllers, which rely significantly on predefined rule sets, by enabling adaptive control in dynamic environments. The ANFIS controller is designed with angle error, voltage disturbances, and torque disturbances as input variables, and outputs compensatory control signals to mitigate the impact of these disturbances. This study introduces a parallel control strategy that integrates EMPC and ANFIS to enhance the transient response and disturbance rejection in gimbal systems, improving both the performance and robustness. The objective of this study is to demonstrate the concept that a parallel EMPC–ANFIS control structure can maintain system performance in the presence of disturbances. To the best knowledge of the authors, this study presents the first experimental application of a parallel EMPC–ANFIS structure on a QUBE-Servo 3 drone gimbal system.

The remainder of this paper is organized as follows. In Section II, a gimbal system is identified to evaluate its performance. In Section III, the EMPC controller is designed and applied to the gimbal system. In Section IV, the ANFIS is designed and the methodology for the parallel integration of ANFIS and EMPC is proposed.

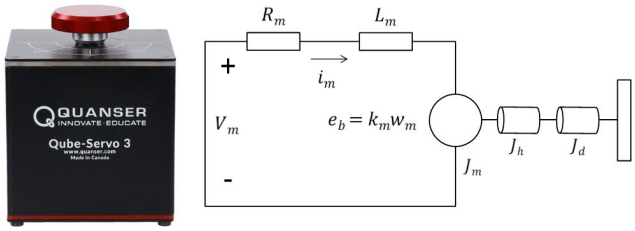

FIGURE 1. Schematic of the motor armature circuit of the QUBE-Servo 3. This diagram shows the circuit schematic of the QUBE-Servo 3 DC motor and load hub. V_m, i_m, and e_b denote the input voltage, current, and back-emf voltage, respectively. R_m and L_m denote the terminal resistance and rotor inductance. The shaft of the DC motor is connected to the load hub. J_h, J_d, and J_m denote the load hub, disk load, and rotor inertia, respectively.

II. GIMBAL MODELING

A. GIMBAL SYSTEM IDENTIFICATION

This study implements a drone gimbal control system using the QUBE-Servo 3 model by Quanser. The QUBE-Servo is a widely used model for evaluating the performance of the control algorithms. For example, Khaimuldin uses QUBE-Servo to validate the performance of a position controller based on model predictive control (MPC) [44]. Adarsh uses QUBE-Servo to implement PD, SMC, and LQR controllers and to experimentally compare their performance [45]. The QUBE-Servo 3 consists of a brushed DC motor equipped with an encoder to measure the angular position, making it an ideal candidate for evaluating the control performance. By adjusting the input voltage, the rotational speed and torque of the DC motor can be controlled, allowing for the implementation of various control algorithms. The DC motor is connected to an inertia disk module, which rotates about a vertical axis. The inertia disk can be precisely controlled, similar to a servo motor, in terms of angle and speed, making it suitable for modeling the yaw-axis motion of a gimbal system. Since the inertia disk operates under a torque-based motor control system analogous to that of a gimbal, it is suitable for estimating the transfer function and analyzing the dynamic responses under inertial loads. Therefore, this study estimates the transfer function of the gimbal system using the QUBE-Servo 3, models the yaw-axis motion, and evaluates the performance of the control algorithms designed to mitigate the effects of inertia and external disturbances. In this study, the sampling period of the control system is set to 0.1 s to ensure reliable communication between the PC and the QUBE-Servo 3.

The Quanser QUBE-Servo 3 is a direct-drive rotary servo system. In this study, the QUBE-Servo 3 is used to verify real-time control performance and represent the yaw-axis motion of the gimbal system. The motor armature circuit schematic is shown in Figure 1, and the corresponding

TABLE 1. QUBE-Servo 3 system parameters.

Symbol	Description	Value
R_m	Terminal resistance	8.4Ω
k_t	Torque constant	$0.042Nm/A$
k_m	Motor back-emf constant	$0.042V/(rad/s)$
J_m	Rotor inertia	4.0×10^{-6} kg·m^2
L_m	Rotor inductance	$1.16mH$
m_h	Load hub mass	$0.0106kg$
J_h	Load hub inertia	0.6×10^{-6} kg·m^2
m_d	Mass of disk load	$0.053kg$
r_d	Radius of disk load	$0.0248m$
K	Steady-state gain	$22.4rad/s/V$
τ	Time constant	$0.15s$

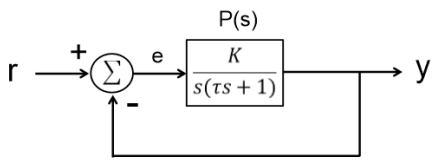

FIGURE 2. Closed-loop transfer function of the position control. This figure shows a second-order system with unity feedback applied to the voltage-to-position transfer function. K denote the steady-state gain, and τ denote the time constant. This block diagram shows a position control system, where the output angle y operates to track the reference input r.

parameters are listed in Table 1. V_m and i_m denote the input voltage and circuit current, respectively. The shaft of the DC motor is connected to the load hub, a metallic disk designed to mount the inertia disk, with the moment of inertia denoted by J_h. An additional disk load with the moment of inertia J_d is attached to the output shaft. The back electromotive force (back-EMF) e_b is generated in opposition to the current flow and depends on the motor shaft velocity w_m and the motor back-EMF constant k_m.

$$P(s) = \frac{\Theta_m(s)}{V_m(s)} = \frac{K}{s(\tau s+1)}, \tag{1}$$

$$\frac{Y(s)}{R(s)} = \frac{\Theta_m(s)}{\Theta_d(s)} = \frac{\frac{K}{\tau}}{s^2 + \frac{1}{\tau}s + \frac{K}{\tau}}. \tag{2}$$

In this study, the voltage-to-position transfer function of the QUBE-Servo 3 is denoted as $P(s)$. The parameters K and τ represent the steady-state gain and time constant of the QUBE-Servo 3 model, respectively. $\Theta_m(s)$ and $V_m(s)$ denote the position of the motor/disk and the input voltage, respectively. Figure 2 shows a block diagram of the closed-loop transfer function for the position control of the QUBE-Servo 3 implemented with unity feedback. This system represents the relationship between the reference input $R(s)$ and the output $Y(s)$ in terms of the rotational angle of the DC motor. In this study, a second-order SISO system is introduced as the plant in order to intuitively understand the design of the EMPC and ANFIS and to emulate the behavior of the gimbal system.

Figure 3 shows the output angle of the DC motor over time in response to a reference input of 1 rad. The simulation is conducted using Matlab/Simulink®. The variation in the output angle from 0 to 1 second demonstrates that the DC motor is significantly affected by inertia. In this study, the objective of the controller design is to reduce the influence of inertia on the DC motor, which represents the yaw-axis motion of the gimbal system, thereby minimizing the rise time, settling time, and overshoot.

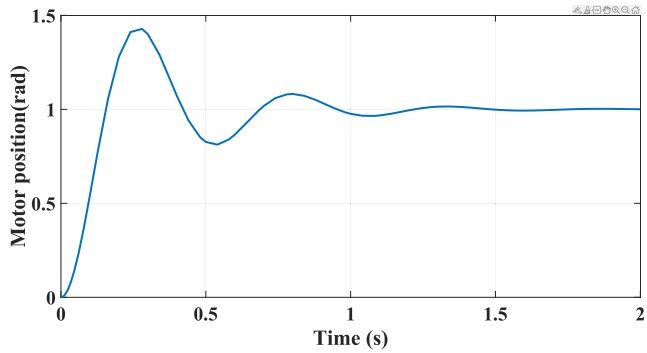

FIGURE 3. Output of the position control. This graph shows the output response of the position control system with a reference input of 1 rad. From 0 to 1 second, the output is unstable. This result shows that the rotational angle of the motor changes due to inertia. In this study, the objective of the controller design is to minimize the effect of inertia on the motor's rotational angle.

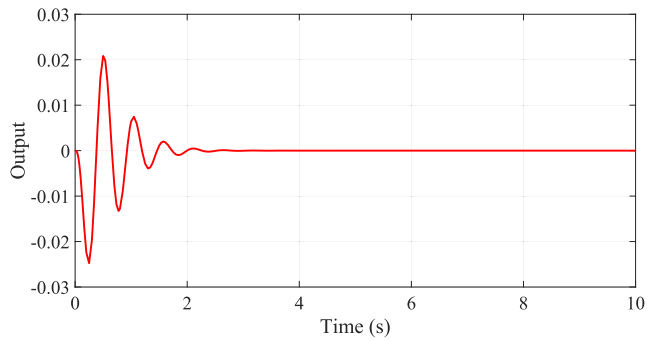

FIGURE 4. Step response error between the hardware and simplified model. Comparison of the step response error between the QUBE-Servo 3 hardware and the simplified second-order model, showing rapid convergence and a small steady-state deviation.

Figure 4 shows the step response error between the QUBE-Servo 3 and simplified second-order model. A small transient response appears in the initial phase, although the error converges to zero after 2 s, and a stable steady state is maintained. This result indicates that the simplified model captures the dominant dynamics of the actual system and is suitable for controller design and simulation validation in experimental environments. The small residual error demonstrates the high consistency between the model and hardware response.

B. PID CONTROLLER DESIGN

In this study, a PID controller is designed to improve the performance of the gimbal system and serve as a baseline for comparison with explicit model predictive control (EMPC). The PID controller generates a control input based on fixed parameters to improve the performance of the control system. The proportional gain (k_p) reduces the rise time, the integral gain (k_i) eliminates the steady-state error, and the derivative

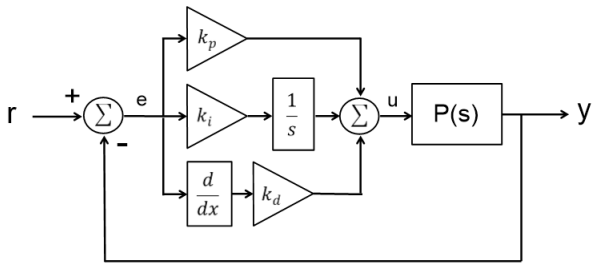

FIGURE 5. Block diagram of the position control with the PID controller. This figure shows the position control block diagram with the PID controller. This PID controller is used for comparison of the control performance with the EMPC. k_p, k_i, and k_d are set to 2.5, 1.5, and 0.1, respectively. k_p reduces rise time, k_i eliminates steady-state error, and k_d decreases overshoot. The PID controller improves the transient performance of the position control system.

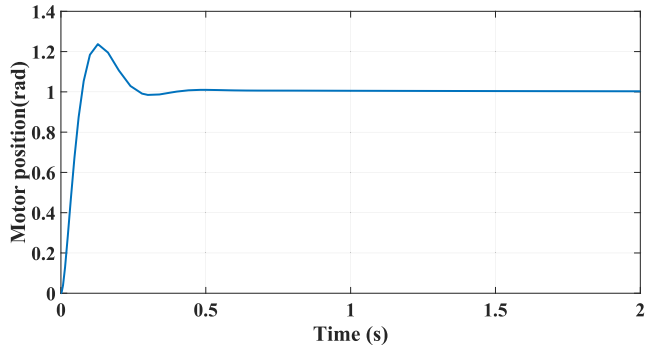

FIGURE 6. Output of the position control with the PID controller. This graph shows the output response of the position control system using a PID controller with a reference input of 1 rad. The change in the output from 0 to 1 second demonstrates that the instability observed in Figure 3 is significantly reduced. This result demonstrates that the PID controller reduces the effect of inertia and improves the performance of the motor rotation.

gain (k_d) decreases the overshoot, thereby improving the overall performance of the gimbal system. In this paper, the values of k_p, k_i, and k_d are set to 2.5, 1.5, and 0.1, respectively. Figure 5 shows the block diagram of the gimbal system with the implemented PID controller.

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau)\, d\tau + k_d \frac{de(t)}{dt}. \tag{3}$$

Figure 6 shows the simulation result of applying the PID controller to the gimbal system with the reference input of 1 rad. As shown in Figure 3, the application of the PID controller reduces the influence of the inertia on the system. However, since the PID controller generates control inputs based on fixed parameters, it has limitations in adapting to changes in the system dynamics. Therefore, the objective of the EMPC design is to reduce the effects of inertia and generate improved control inputs by predicting future behavior based on the dynamic model of the gimbal system, thereby outperforming the PID controller.

III. EXPLICIT MODEL PREDICTIVE CONTROL

A. CONCEPT OF EXPLICIT MODEL PREDICTIVE CONTROL

The EMPC framework is based on a general linear discrete-time system, which can be expressed as follows:

$$\begin{aligned} x(k+1) &= Ax(k) + Bu(k), \\ y(k) &= Cx(k). \end{aligned} \tag{4}$$

$x(k) \in \mathbb{R}^n$ denotes the state vector, $u(k) \in \mathbb{R}^m$ denotes the control input and $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times m}$, and $C \in \mathbb{R}^{p\times n}$ denote the system matrices. The optimization problem of MPC, formulated as a quadratic programming (QP) problem over a prediction horizon N, is given as follows:

$$\min_U \quad J(x(k), U) = \frac{1}{2} U^T H U + x(k)^T F^T U,$$

$$\text{subject to:} \quad GU \le W + Ex(k), \tag{5}$$

$$U = \left[u(k)^\top \cdots u(k+N-1)^\top\right]^\top \in \mathbb{R}^{Nm}, \tag{6}$$

$$H = H^\top \succ 0. \tag{7}$$

U denotes the predicted input vector, H denotes the positive definite matrix associated with the cost function, and F, G, W, E denote the offline matrices constructed based on the prediction model and constraints. This optimization problem is a typical QP problem that minimizes a quadratic cost function subject to linear constraints. As the mp-QP problem is strictly convex ($H \succ 0$), the optimizer is unique for any parameter vector z. As a result, the explicit solution is piecewise continuous across region boundaries. Although the QP problem is commonly solved online at each time step, EMPC solves the QP problem offline for all possible states $x(k)$. EMPC partitions the state space and derives piecewise control laws for each region. The state space is partitioned as follows:

$$\mathcal{R}_i = \left\{x \in \mathbb{R}^n \mid H_i x \le K_i\right\}, \quad i = 1, 2, \ldots, N_r. \tag{8}$$

R_i denote the polyhedral region in the state space. H, and K denote the inequality matrix and vector that define each region. N_r denotes the total number of regions. In each partitioned region, a different control law is applied to compute the control input. The optimal control input for each region is expressed as a linear function of the state as follows:

$$u(k) = F_i x(k) + G_i, \quad \text{if } x(k) \in \mathcal{R}_i. \tag{9}$$

F denotes the state feedback gain. G denotes the constant offset vector. The mp-QP parameter vector is defined as

$$z = \left[x^\top \; r \; u_{\text{prev}}\right]^\top = \begin{bmatrix} x_1 \\ x_2 \\ r \\ u_{\text{prev}} \end{bmatrix}, \tag{10}$$

which corresponds to the classification vector used for region identification. This vector determines the active region for the piecewise-affine control law

$$\Delta u = F_i z + G_i, \quad \text{if } H_i z \le K_i. \tag{11}$$

Here, $mv = u_{\text{prev}}$ denotes the previously applied control input, which is fixed during the current computation and thus

prevents any algebraic loop in the region-selection process. In conclusion, EMPC computes the optimal control input by solving the optimization problem offline, partitioning the state space based on the range of state variables, and applying a different control law in each region. The stability of EMPC has been theoretically verified in previous studies [23], [25], [26].

B. EMPC-BASED CONTROL ALGORITHM

This subsection presents the EMPC controller design algorithm based on the concept of EMPC. First, the QP problem is formulated based on the system model representing the plant and the associated constraints. The state space is then partitioned using polyhedral analysis techniques, such as multiparametric programming. To compute the control input, the region corresponding to the current state is determined by applying the region identification conditions, followed by the application of the corresponding control law. This result is stored in the following form:

$$\begin{cases} \mathcal{R}_i : H_i x \leq K_i \\ u(k) = F_i x + G_i \end{cases} \quad i = 1, 2, \ldots, N_r. \tag{12}$$

During real-time control, EMPC measures the current state $x(k)$ and identifies the index i such that $x(k) \in R_i$

$$H_i x(k) \leq K_i. \tag{13}$$

Then, the control law corresponding to the identified region is applied to compute the control input.

$$u(k) = F_i x(k) + G_i. \tag{14}$$

At this stage, the polyhedral search algorithm is optimized, enabling fast region identification by the EMPC controller, which makes the overall control strategy suitable for real-time implementation.

C. EMPC CONTROLLER DESIGN

The transfer function of the gimbal system is represented as a second-order system.

$$y(t) = \frac{149.33}{s^2 + 6.67s + 149.33} u(t). \tag{15}$$

Since EMPC operates based on a linear discrete-time system, it is necessary to convert the continuous-time system into a discrete-time system. The sampling period is important for reflecting the real-time characteristics of the control system. The sampling period is set to $T_s = 0.1$ s considering the USB communication environment between the QUBE-Servo 3 and PC. The equivalent discrete-time state-space model is represented as follows:

$$\begin{aligned} x(t+1) &= \begin{bmatrix} 0.0882 & -0.5249 \\ 0.8999 & 0.4634 \end{bmatrix} x(t) + \begin{bmatrix} 0.2250 \\ 0.2300 \end{bmatrix} u(t), \\ y(t) &= \begin{bmatrix} 0 & 2.3333 \end{bmatrix} x(t). \end{aligned} \tag{16}$$

MPC predicts future control inputs based on the system's dynamic model and applies the optimized inputs. In this study, EMPC is configured with the following settings. The prediction horizon is set to N_p=10. The prediction horizon means that EMPC predicts the system output for the next ten steps from the current time. The control horizon is set to N_c=3. Within this horizon, the control inputs are optimized for three steps, and it is assumed that the inputs remain constant afterward. The manipulated variable is constrained by $-1 \leq u(t) \leq 1$, indicating the maximum allowed input voltage that can be physically applied to the system. This constraint prevents potential damage caused by excessive control input.

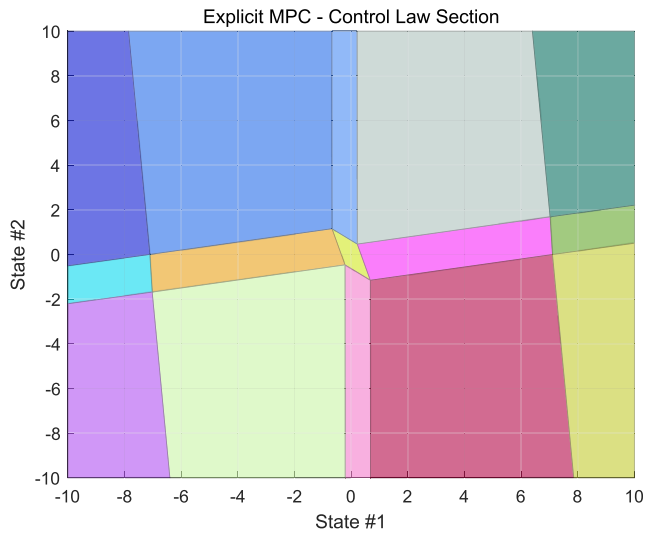

FIGURE 7. Critical regions of EMPC.

The ranges of the state variables x_1 and x_2 are defined as follows:

$$\begin{aligned} -10 \leq x_1(t) \leq 10, \\ -10 \leq x_2(t) \leq 10. \end{aligned} \tag{17}$$

The state variables are defined to cover the expected operating range of the system, preventing deviations caused by abnormal inputs or outputs.

The range of the reference input is defined as follows:

$$-2 \leq r(t) \leq 2. \tag{18}$$

This represents the valid range of target values that the control system is expected to track.

The range of the control input u during EMPC generation is defined as follows:

$$-1.1 \leq u(t) \leq 1.1 \tag{19}$$

This range is wider than the one defined in the actual controller to alleviate the potential numerical instability near the boundaries of the EMPC regions and to allow the control laws to be defined over a broader input range. QUBE-Servo 3 operates in a voltage range of ±10 V. To prevent potential hardware damage, experiments are conducted under low voltage conditions and control input constraints are defined accordingly(19).

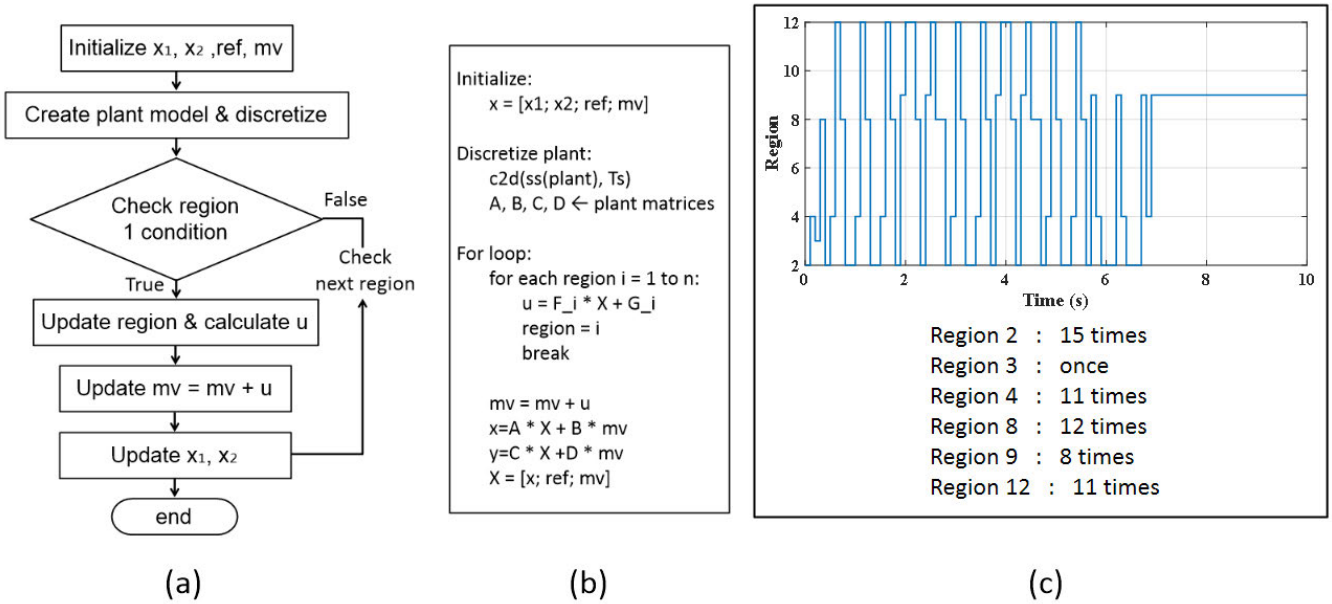

FIGURE 8. EMPC controller algorithm and region application in the simulation. (a) and (b) show the flowchart and pseudocode of the EMPC controller algorithm, respectively. (c) shows the changes in the regions identified by the EMPC controller applied to the drone gimbal system over 10 seconds. This graph demonstrates that EMPC improves the system's performance by computing the optimal control inputs at each sampling step based on the gimbal system's current state.

Region 2

$$\begin{bmatrix} 0.87 & 0.24 & -0.40 & -0.004 \\ -0.13 & 0.75 & 0.64 & -9.04\times10^{-15} \\ 0.15 & 0.007 & 0.98 & -1.12\times10^{-14} \\ 0.13 & -0.75 & -0.64 & 9.04\times10^{-15} \\ -0.15 & -0.007 & -0.98 & 1.12\times10^{-14} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} x \leq \begin{bmatrix} -0.30 \\ 0.96 \\ 0.99 \\ 0.32 \\ 1.12 \\ 2 \\ 1.1 \\ 10 \\ 1.1 \end{bmatrix}$$

$$u = \begin{bmatrix} -1.11\times10^{-16} & 5.55\times10^{-17} & 8.88\times10^{-16} & -0.99 \end{bmatrix} x + 1$$

Region 4

$$\begin{bmatrix} 0.02 & 0.04 & -0.99 & -0.0008 \\ -0.94 & -0.25 & 0.20 & 0.004 \\ 0.57 & 0.74 & 0.34 & -0.003 \\ 0.94 & 0.25 & -0.20 & -0.004 \\ -0.57 & -0.74 & -0.34 & 0.003 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -1 \end{bmatrix} x \leq \begin{bmatrix} -0.98 \\ 0.09 \\ 0.65 \\ 0.56 \\ 0.29 \\ 2 \\ 1.1 \\ 1.1 \end{bmatrix}$$

$$u = \begin{bmatrix} -2.84 & -0.78 & 0.61 & -0.98 \end{bmatrix} x + 0.70$$

Region 8

$$\begin{bmatrix} 0.82 & 0.0004 & -0.55 & -0.003 \\ 0.13 & -0.75 & -0.64 & 9.04\times10^{-15} \\ 0.15 & 0.01 & 0.98 & -1.12\times10^{-14} \\ -0.15 & -0.01 & -0.98 & 1.12\times10^{-14} \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} x \leq \begin{bmatrix} -0.56 \\ -0.96 \\ 0.99 \\ 1.11 \\ 10 \\ 2 \\ 1.10 \\ 10 \\ 1.1 \end{bmatrix}$$

$$u = \begin{bmatrix} 1.11\times10^{-16} & 4.16\times10^{-17} & -8.88\times10^{-16} & -0.99 \end{bmatrix} x + 1$$

Region 12

$$\begin{bmatrix} -0.57 & -0.74 & -0.34 & 0.003 \\ -0.01 & -0.01 & -0.99 & -0.0006 \\ -0.91 & -0.002 & 0.39 & 0.004 \\ 0.91 & 0.002 & -0.39 & -0.004 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -1 \end{bmatrix} x \leq \begin{bmatrix} -0.65 \\ -1.005 \\ 0.39 \\ 0.41 \\ 10 \\ 2 \\ 1.1 \\ 1.1 \end{bmatrix}$$

$$u = \begin{bmatrix} -2.24 & -0.006 & 0.96 & -0.99 \end{bmatrix} x + 0.02$$

FIGURE 9. Region boundaries and control input computation. This figure shows the F, G, H, and K matrices of the defined regions.

The region of EMPC is determined by the matrices H and K (8), (12), (13). The H and K matrices are indirectly derived from the boundary equations that define the valid region of the state space, based on the active set of the

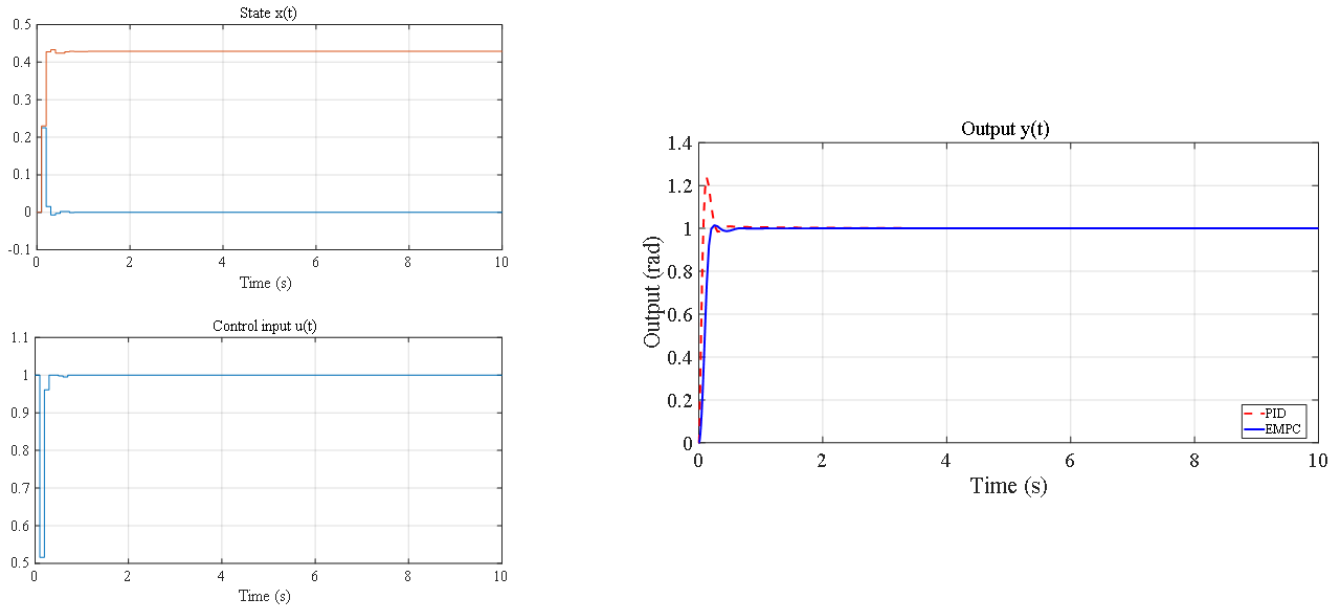

FIGURE 10. Simulation of the gimbal system using the EMPC controller. The state $x(t)$ graph shows the variation of the gimbal system's state variables over time. The control input $u(t)$ graph shows how the control input generated by EMPC changes over time. The output $y(t)$ graph shows the change in the output angle of the gimbal system using the EMPC control. The solid and dashed lines represent the performance of the gimbal system controlled by EMPC and PID, respectively. The graph confirms that EMPC effectively reduces the overshoot caused by the motor inertia more than PID. In conclusion, the drone gimbal system can achieve a more precise control of the camera's shooting angle using EMPC.

QP solution (5). The control input in a specific region is determined by the matrices F and G (9), (12), and (14). The F and G matrices are the region-specific feedback gain and offset term, respectively, obtained during the derivation of the QP solution (5). In this study, the F, G, H, and K matrices are computed using the Model Predictive Control Toolbox in Matlab/Simulink® [46]. Figure 7 shows the critical region derived from EMPC. Each colored region represents an area in the state space where a unique control law is applied. These regions are computed based on the discrete-time system and the defined constraints. By prepartitioning the state space into such regions, EMPC computes the control input using precomputed values, thereby eliminating the need to solve an online optimization problem during real-time operation. In this study, the EMPC applied to the drone gimbal system identified 25 regions.

The controller for the drone gimbal system is designed by implementing the EMPC algorithm through programming. The designed controller operates independently without relying on the Model Predictive Control Toolbox of Matlab/Simulink® and is applied to the drone gimbal system. Figure 8 shows the algorithm of the controller designed based on the concept of EMPC and the regions applied in the actual simulation. Figure 8 (a) and (b) show the flowchart and pseudocode of the EMPC controller, respectively. The EMPC controller starts operating by receiving the state variables, reference input, and initial control input, known as manipulated variable (mv). The EMPC controller receives the plant model of the system and the sampling period T_s and then discretizes the system. In this process, critical regions are generated, and the F, G, H, and K matrices are computed. Once the initial controller setup is completed, the controller computes the control input at every sampling period. The controller first determines which region the current state variables belong to using the H and K matrices. Then, the control input is computed based on the control law associated with that region, defined by the F and G matrices. The control input for the next step is updated by adding the computed control input u to the current manipulated variable. The state variables for the next step are then computed using the updated control input. The EMPC controller repeats this process at each step, continuously calculating and updating the optimal control input to improve the system's performance. 8 (c) shows the regions applied to the drone gimbal control system using the EMPC controller. The simulation runs from 0 to 10 seconds, during which the control input is computed within six different regions. Regions 2, 3, 4, 8, 9, and 12 are applied 15, 1, 11, 12, 8, and 11 times, respectively. The simulation result confirms that the regions visited during system operation are concentrated within the 25 critical regions. Therefore, the calculated number of regions sufficiently covers the actual operating range of the controller and is appropriate for the complexity level of the proposed system. This graph demonstrates how EMPC enhances the system's performance by computing the optimal control input at each sampling period based on the gimbal system.

Figure 9 shows the range of regions applied in the simulation and the control input equations calculated within each region. Each region is defined based on the range of the

TABLE 2. Input constraint and saturation metrics.

max $\|u\|$	max $\|\Delta u\|$	Saturation counts
1.0000	0.4838	0

state variables at a given time. The state variable values are classified using the H and K matrices, and the control input within each region is calculated using the F and G matrices.

Figure 10 shows the simulation results of applying EMPC to the gimbal system. The state $x(t)$ graph shows how the state variables of the gimbal system change over time. The control input $u(t)$ graph shows how the control input generated by EMPC varies with time. The output $y(t)$ graph shows the change in the output angle of the gimbal system under EMPC control. The solid and dashed lines represent the performance of the gimbal system controlled by EMPC and PID, respectively. The gimbal system with EMPC significantly reduced the overshoot caused by inertia more than the system with PID. This result shows that the performance of the gimbal system with EMPC is superior to that of the system with PID. Therefore, the gimbal system of the drone can achieve a more precise control of the camera's shooting angle through EMPC.

Table 2 shows the max $|u|$, max $|\Delta u|$, and saturation counts. max $|u|$ and max $|\Delta u|$ represent the maximum control input and the maximum rate of change, respectively. This result demonstrates that the EMPC operates stably within the control input range(19).

IV. ADAPTIVE NEURO-FUZZY INFERENCE SYSTEM

A. LOAD DISTURBANCE

Evaluating a system's robustness to unknown disturbances is a critical aspect of control system evaluation [47], [48]. In this study, the voltage and torque disturbances are loaded. In a drone's gimbal system, voltage disturbances are caused by battery voltage discharge, PWM waveform noise, and DC-DC converter effects. Torque disturbances are the result of external impacts, load variations, and mechanical imperfections, such as backlash or gear errors. The following relationship holds among the terminal resistance, motor back-emf constant, and equivalent moment of inertia.

$$\frac{R_m}{k_m} = \frac{\tau}{KJ_{eq}}. \tag{20}$$

R_m, k_m, and J_{eq} denote the terminal resistance, motor back-emf constant, and equivalent moment of inertia, respectively. K and τ represent the steady-state gain and time constant of the drone gimbal system, respectively. Figure 11 shows the block diagram in which the relationship in Equation (20) is applied to the system model in Figure 2 to load the disturbances.

d_V and d_T denote the voltage and torque disturbance, respectively. In this study, the disturbance ranges are determined based on real-world operating conditions. The voltage disturbance range is set from -0.1 to $+0.1V$, considering

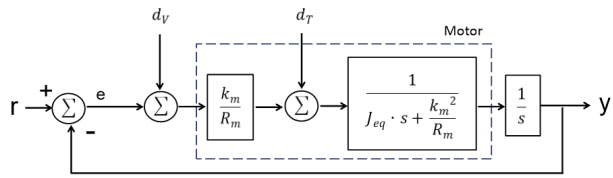

FIGURE 11. Block diagram with the disturbance input. This block diagram is a modified version of Figure 11 based on the relationship defined in Equation (20). R_m, k_m, and J_{eq} denote the terminal resistance, motor back-emf constant, and equivalent moment of inertia, respectively. K and τ denote the steady-state gain and time constant of the drone gimbal system, respectively. d_V and d_T denote the voltage and torque disturbances, respectively.

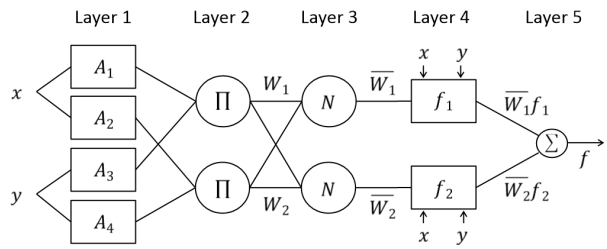

FIGURE 12. ANFIS layer architecture.

the PWM noise and battery discharge conditions. The torque disturbance range is set from -0.001 to $+0.001Nm$, considering the vibrations caused by wind and impact, as well as the load variations. The values of R_m, k_m, and J_{eq} are 8.4Ω, $0.042V/(rad/s)$, and 3.34×10^{-5} kg·m^2, respectively. This block diagram allows us to analyze the impact of disturbances on the gimbal system and evaluate the disturbance rejection performance of the designed controller.

B. CONCEPT OF ANFIS

The Adaptive neuro-fuzzy inference system (ANFIS) is a model that combines a fuzzy inference system with an artificial neural network, featuring a structure that automatically learns the parameters of fuzzy rules from data. ANFIS effectively approximates complex nonlinear relationships between inputs and outputs by leveraging both the interpretability of fuzzy systems and the learning capability of neural networks. Typically based on the Takagi–Sugeno-type fuzzy model, ANFIS is structured as a multilayered neuro-fuzzy network, with each layer performing a specific operation [49]. The stability of ANFIS has been demonstrated in previous research [30].

1) LAYER 1: FUZZIFICATION LAYER

In Layer 1, the membership degree for each input is calculated. For inputs x and y, the outputs of this layer are as follows.

$$O_{1,i} = \mu_{A_i}(x), \quad O_{1,j} = \mu_{B_j}(y). \tag{21}$$

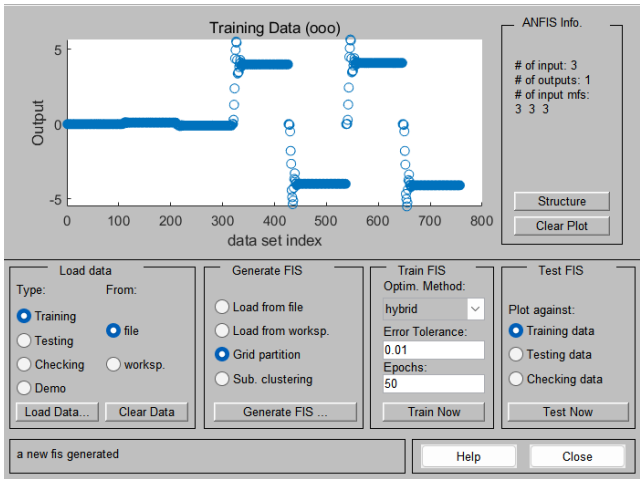

FIGURE 13. ANFIS training process. The input variables of ANFIS are error, d_V, and d_T, and the output variable is u_{ANFIS}. The ANFIS training data is obtained from scenarios A to G in Table 3. The input and output variables are defined using three triangular membership functions. The number of training epochs is set to 50, and the training stops when the error reaches 0.01.

μ_{A_i} and μ_{B_j} denote the membership functions for the input variables x and y, typically represented by Gaussian, triangular, or bell-shaped functions. The parameters in this layer are called the premise parameters and are adjusted during the learning process.

2) LAYER 2: RULE LAYER

In Layer 2, the firing strength of each possible rule is calculated. Each rule is computed using an AND operation as follows.

$$O_{2,k} = w_k = \mu_{A_i}(x) \cdot \mu_{B_j}(y). \tag{22}$$

w_k denotes the firing strength of the k-th fuzzy rule. This layer performs only fixed operations and does not contain any trainable parameters.

3) LAYER 3: NORMALIZATION LAYER

In Layer 3, the firing strengths of all the rules are normalized to represent the relative contribution of each rule among all the rules.

$$O_{3,k} = \bar{w}_k = \frac{w_k}{\sum_{i=1}^{N} w_i}. \tag{23}$$

This normalization reflects the relative importance of each rule and is used in the computations of subsequent layers.

4) LAYER 4: DEFUZZIFICATION LAYER

In Layer 4, a linear subsequent function is computed for each rule, and a weighted sum is calculated using the normalized firing strengths.

$$O_{4,k} = \bar{w}_k \cdot f_k = \bar{w}_k \cdot (p_k x + q_k y + r_k). \tag{24}$$

f_k denotes the consequential part of the k-th rule, and p, q, and r denote the trainable consequent parameters.

TABLE 3. ANFIS training scenario.

	Input			Output
scenario	error (r-y)	d_V	d_T	u_{comp}
A	e_A	0	0	$u_{comp,A}$
B	e_B	0.1	0	$u_{comp,B}$
C	e_C	-0.1	0	$u_{comp,C}$
D	e_D	0	0.02	$u_{comp,D}$
E	e_E	0	-0.02	$u_{comp,E}$
F	e_F	0.1	0.02	$u_{comp,F}$
G	e_G	-0.1	-0.02	$u_{comp,G}$

5) LAYER 5: OUTPUT LAYER

Layer 5 sums the outputs of all the rules to generate the final output of the system.

$$O_5 = \sum_{k=1}^{N} \bar{w}_k \cdot f_k. \tag{25}$$

This output is the inference result of ANFIS, which is compared with the actual output of the target system to calculate the error.

ANFIS adjusts its parameters using a hybrid learning algorithm [50]. This structure has clearly defined operations at each layer, ensuring both interpretability and learning efficiency. Training is generally performed based on input–output data pairs, with the parameters being iteratively updated to minimize the error between the network's overall output and the actual output.

C. ANFIS DESIGN

ANFIS is designed to learn from the training data. The inputs to the ANFIS controller are the error between the reference input and the output, voltage disturbance, and torque disturbance. The output is a control input that compensates for these disturbances. In this study, the output of ANFIS is determined by u_{comp}, u_{ideal}, and $u_{disturbed}$. u_{ideal} and $u_{disturbed}$ represent the control inputs when there is no disturbance and when there is a disturbance, respectively. u_{comp} represents the compensating control input that compensates for the disturbance. This value corresponds to the ideal output of the ANFIS. The relationship among u_{comp}, u_{ideal}, and $u_{disturbed}$ is given as follows.

$$u_{comp} = u_{ideal} - u_{disturbed} \tag{26}$$

The ANFIS rule base learns the difference in control input caused by disturbances through training data, where the inputs are error, d_V, and d_T, and the output is u_comp. ANFIS estimates the magnitude of the disturbance based on the input variables and generates a control input u_{ANFIS} to compensate for the disturbance. u_{ANFIS} represents the control input produced by ANFIS during the actual simulation, learned based on the relationship between the disturbances and u_{comp}. When there is no disturbance, u_{ANFIS} is zero.

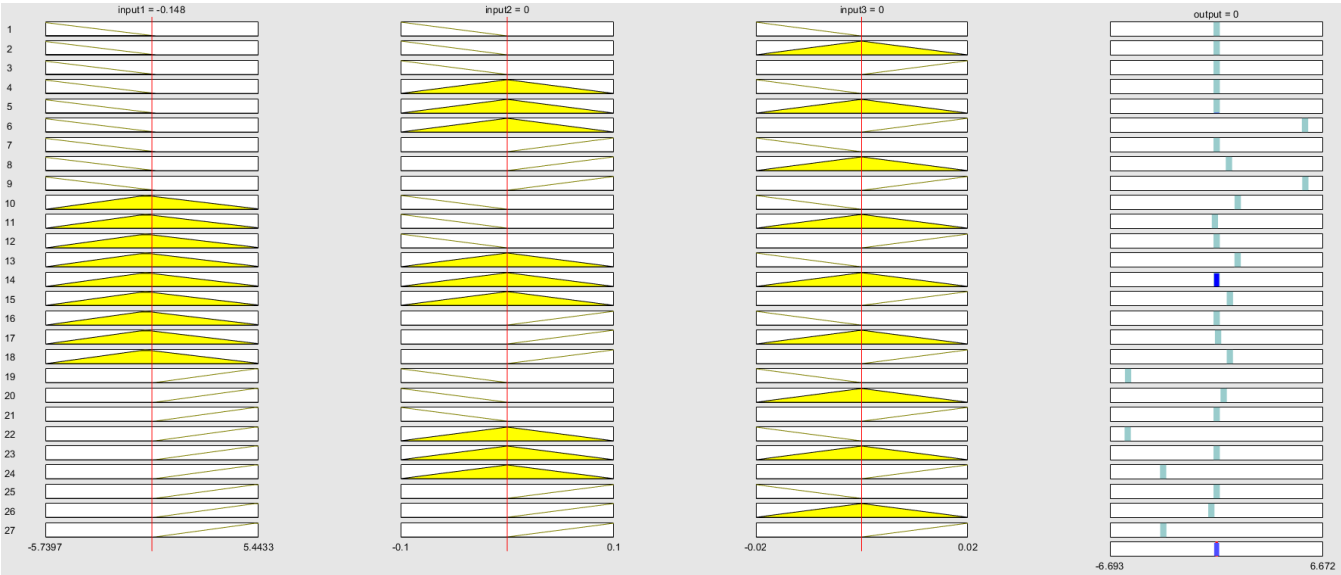

FIGURE 14. Fuzzy rules of ANFIS. This figure shows the input and output membership functions and the activated fuzzy rules of ANFIS. Input 1, 2, and 3 correspond to the error, d_V, and d_T, respectively.

When there is a disturbance, u_{ANFIS} is computed at a specific value to minimize the variation in the output caused by the disturbance.

Figure 13 shows the ANFIS design process using Matlab/Simulink®. The input variables of ANFIS are error, d_V, and d_T, and the output variable is u_{ANFIS}. The ANFIS rule base is trained to minimize the output error caused by disturbances based on the relationship between the input variables and u_{comp}. Table 3 presents the ANFIS training scenarios. From scenarios A to G, ANFIS learns the differences in the control input caused by disturbances through error, d_V, d_T, and u_{comp}. d_V is set in the range of -0.1 to 0.1, and d_T is set in the range of -0.02 to 0.02. The input and output variables are defined using three triangular membership functions. The number of training epochs is set to 50, and the training stops when the error reaches 0.01. Based on the training data, ANFIS determines the sign and magnitude of the compensating control input. When the torque disturbance is large and significantly impacts the output, ANFIS generates a large compensating control input. When the voltage disturbance is positive, ANFIS outputs a negative compensating control input to maintain the voltage level. In other words, ANFIS adjusts the magnitude and sign of u_{ANFIS} according to the size and direction of the disturbance to minimize its effect on the system output.

Figure 14 visualizes the membership functions and rule activation states of the ANFIS. Inputs 1, 2, and 3 correspond to the error, d_V, and d_T, respectively. The membership functions of the three input variables are defined in a triangular shape. The yellow-shaded regions represent the rules activated by the current input values. The output section shows the distribution of output weights corresponding to each rule, visually illustrating how ANFIS generates the output for a given combination of input variables. This visualization confirms that the ANFIS has effectively learned the appropriate rules for each input range.

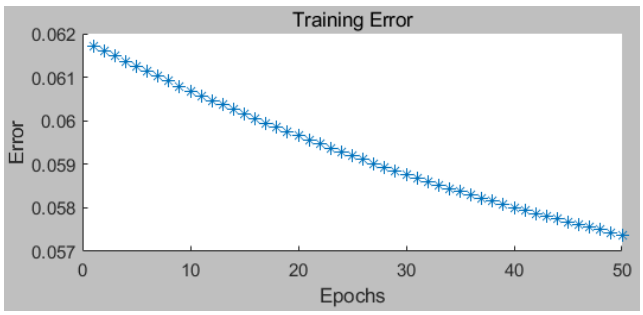

FIGURE 15. ANFIS training error. The training error gradually decreases over 50 epochs. The error tolerance is set to 0.01 to prevent overfitting and maintain generalization performance.

Figure 15 shows the variation of the training error during the learning process of ANFIS. Training is performed over 50 epochs, and the error gradually decreases from 0.062 to 0.057. This result indicates that the ANFIS stably learns the relationship between input and output variables. By setting the error tolerance to 0.01, overfitting is prevented to the training data and generalization performance is maintained.

Figure 16 shows the network structure of ANFIS. The input layer receives three input variables: error, voltage disturbance, and torque disturbance. Each input is fuzzified through three membership functions. In the rule layer, 27 fuzzy rules are activated based on all possible combinations of input membership functions. The output layer aggregates the weighted outputs of these rules to compute the final control signal. This structure simultaneously demonstrates

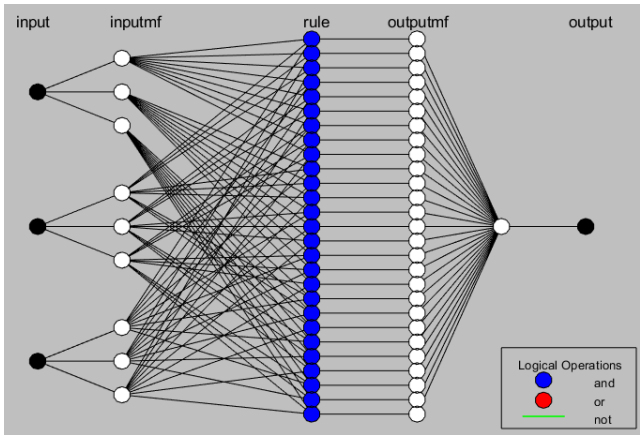

FIGURE 16. ANFIS network architecture. The layered structure shows how input membership functions, fuzzy rules, and output nodes interact to generate the final output.

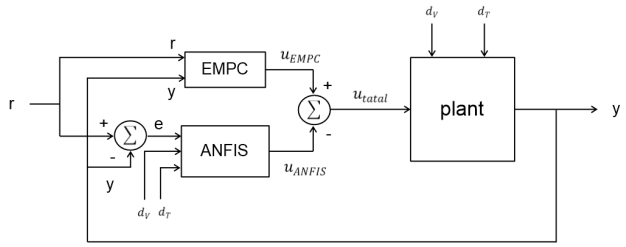

FIGURE 17. Parallel structure of EMPC and ANFIS. EMPC improves the performance of the gimbal system by reducing the effects of inertia. The inputs to EMPC are the reference input and the system output, and its output is u_{EMPC}. ANFIS compensates for disturbances to minimize the output error caused by disturbances. The inputs to ANFIS are error, d_V, and d_T, and the output is u_{ANFIS}. The final control input, u_{total}, is obtained by subtracting u_{ANFIS} from u_{EMPC}. The EMPC and ANFIS parallel controllers work by calculating the optimal control input u_{EMPC} at each sampling step and then reducing it by the disturbance-compensating input u_{ANFIS}.

the interpretability of fuzzy rules and the learning capability of the neural network.

D. PARALLEL STRUCTURE OF ANFIS AND EMPC

To improve the performance of the drone gimbal system and compensate for disturbances, a parallel structure of EMPC and ANFIS is introduced. Figure 17 shows the block diagram of the parallel structure of EMPC and ANFIS. u_{EMPC}, u_{ANFIS}, and u_{total} represent the control inputs of EMPC, ANFIS, and the overall controller, respectively.

EMPC acts as the main controller, which improves the performance of the gimbal system by reducing the effects of inertia. The inputs to the EMPC are the reference input and the system output, and its output is u_{EMPC}. ANFIS acts as a secondary controller, which compensates for disturbances to minimize their impact on the system output. The inputs to ANFIS are error, d_V, and d_T, and the output is u_{ANFIS}. The final control input, u_{total}, is obtained by subtracting u_{ANFIS} from u_{EMPC}. EMPC divides the control regions in advance based on the dynamic model of the system,

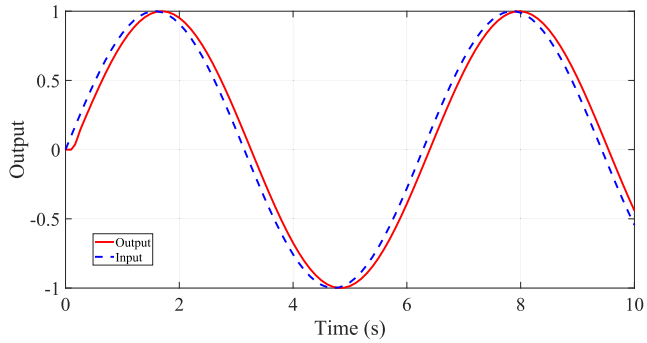

FIGURE 18. BIBO stability analysis. The dashed and solid lines represent input and output signals, respectively. This figure shows one example in which the overall system satisfies BIBO stability.

meaning it cannot account for disturbances during real-time simulation. In contrast, ANFIS generates a compensating control input u_{ANFIS} based on the relationship between the control input and the output. Therefore, the final control input u_{total} is obtained by subtracting u_{ANFIS} from u_{EMPC}. The parallel structure of the EMPC and ANFIS controller operates by computing the optimal control input u_{EMPC} at each sampling step and subtracting the disturbance-compensating input u_{ANFIS}. This parallel approach, which adds ANFIS alongside EMPC, predicts the next output at each sampling step and generates an additional control input to compensate for the disturbances. This improves the performance and closed-loop stability of the system, particularly in discrete-time systems. To the best knowledge of the authors, this study presents the first experimental application of a parallel EMPC–ANFIS structure on a QUBE-Servo 3 drone gimbal system. To implement a parallel structure with ANFIS, which estimates and compensates for disturbances at each sampling step, the real-time computational complexity of the main controller must be considered. EMPC is well-suited as the main controller in this parallel setup because it uses precomputed control laws based on the critical regions, resulting in a low computational cost. When ANFIS compensates for disturbances, the overall system must operate stably within a predictable range. Unlike discrete-time PID controllers without constraints, which apply the computed control input regardless of its magnitude, EMPC predicts system behavior using a plant model and applies control inputs within defined constraints, ensuring greater stability. In summary, EMPC provides optimal control by predicting future states and enforcing constraints. This makes EMPC highly suitable as the main controller, especially when paired with ANFIS for disturbance compensation, as it ensures both high stability and computational efficiency. Figure 18 shows the BIBO (bounded-input bounded-output) stability analysis of the parallel EMPC–ANFIS structure. The dashed and solid lines represent input and output signals, respectively. The ANFIS is constructed using normalized membership functions and a finite rule base, which ensures bounded outputs for bounded inputs. Under appropriate constraints, the EMPC guarantees

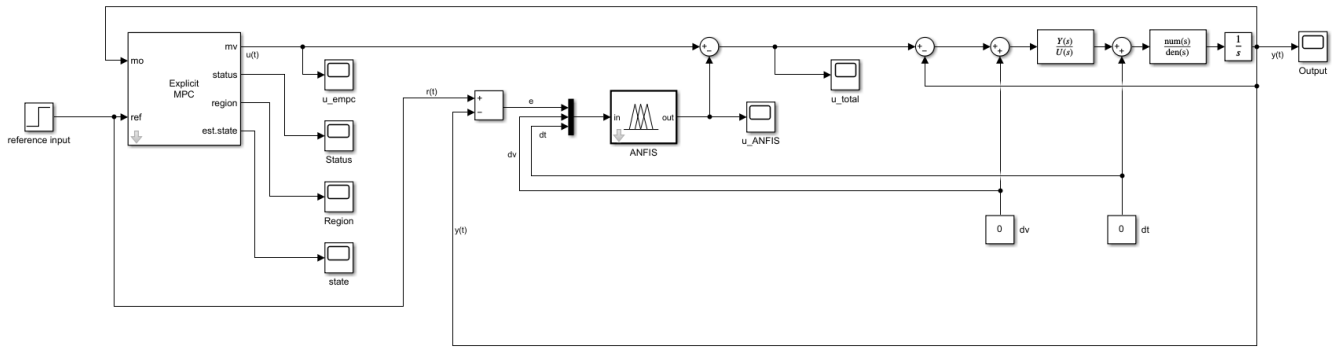

FIGURE 19. Parallel structure of EMPC and ANFIS in Matlab/Simulink®.

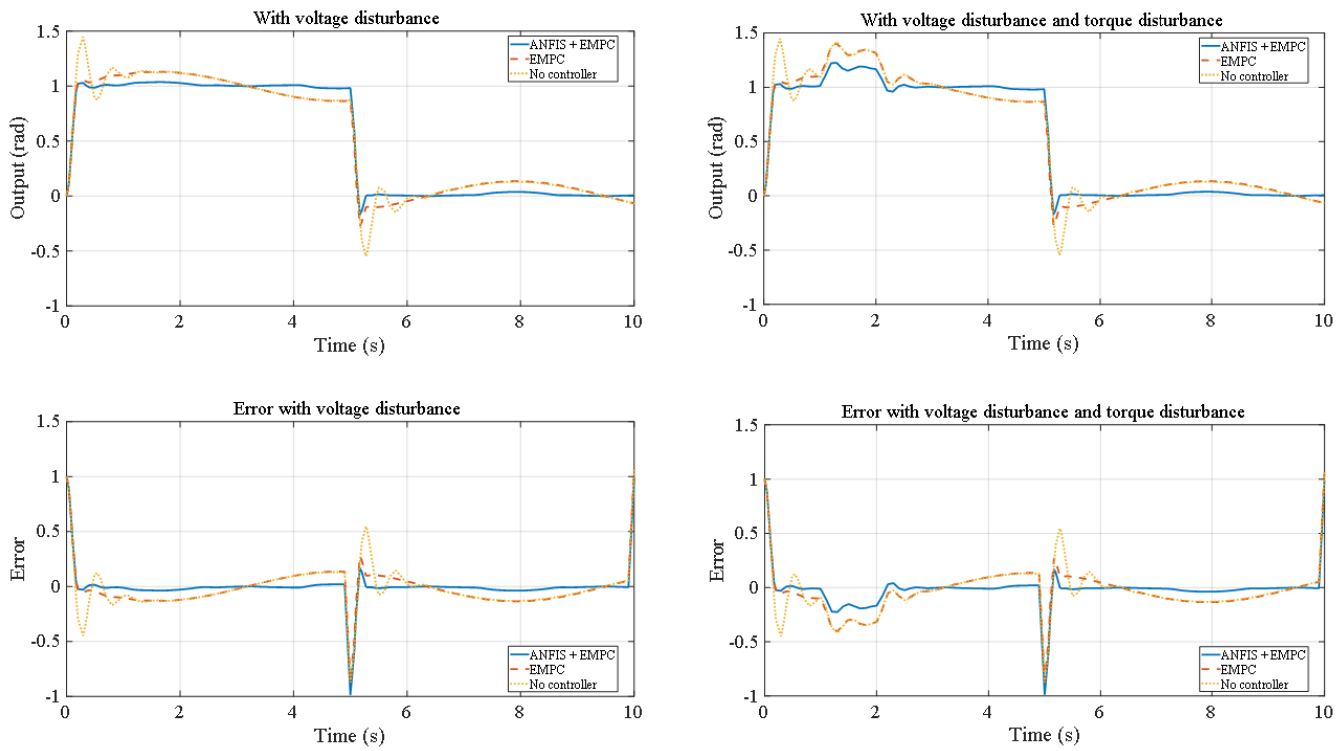

FIGURE 20. Simulation results. The graphs compare the system output and error for three cases: no controller, EMPC, and the proposed parallel structure. When only a voltage disturbance is applied, EMPC mitigates inertia-induced effects, while the parallel structure further improves tracking accuracy and stability. Under combined voltage and torque disturbances, the parallel controller effectively estimates and compensates for external disturbances, achieving the smallest tracking error and minimal output distortion.

bounded states and outputs for bounded inputs. Therefore, the parallel combination of EMPC and ANFIS satisfies the overall BIBO stability of the system and provides a structural stability guarantee independent of specific parameter tuning.

Figure 19 shows the block diagram of the gimbal system with a parallel structure of EMPC and ANFIS applied in Matlab/Simulink®. Figure 20 shows the simulation results of the block diagram in Figure 19 with the introduction of a disturbance. The graphs titled "With voltage disturbance" and "Error with voltage disturbance" show the output and error of the gimbal system, respectively, in the presence of a voltage disturbance. The graphs titled "With voltage disturbance and torque disturbance" and "Error with voltage disturbance and torque disturbance" show the output and error of the gimbal system, respectively, when both voltage and torque disturbances are present. The voltage disturbance is modeled as a sine wave with an amplitude of 0.13 and a frequency of 1 rad/s, representing PWM noise and battery discharge conditions. The torque disturbance is modeled as a pulse wave with an amplitude of 0.001 applied from

Controller	IAE	ITAE	Tr	OS
{'ANFIS+EMPC' }	0.57098	2.0648	0.1197	22.493
{'EMPC' }	1.2562	5.3005	0.11914	40.524
{'No controller'}	1.4002	5.6153	0.106	44.824

FIGURE 21. Quantitative performance metrics of each controller. IAE, ITAE, T_r, and OS represent the integral of absolute error, the integral of time-weighted absolute error, the rise time, and the percent overshoot, respectively.

1 to 2 seconds to simulate the vibrations caused by external impacts. The reference input represents a scenario in which the yaw-axis motor of the gimbal system rotates by 1 rad and then returns to its original position. In the "With voltage disturbance" and "With voltage disturbance and torque disturbance" graphs, the solid, dashed, and dotted lines represent the output of the parallel structure controller, EMPC, and uncontrolled system, respectively. In the "Error with voltage disturbance" and "Error with voltage and torque disturbance" graphs, the solid, dashed, and dotted lines represent the error between the reference input and the output for the parallel structure controller, EMPC, and uncontrolled system, respectively. When no controller is applied to the gimbal system, the effect of inertia is most prominent. This indicates that EMPC improves system performance by minimizing the influence of inertia. Under a voltage disturbance, the output of the system without the controller and with the EMPC controller exhibits a sine wave pattern. In contrast, the system with the parallel structure controller combining EMPC and ANFIS minimizes the influence of the sine wave. This result shows that ANFIS estimates the voltage disturbance and compensates for it, allowing the system to maintain a stable output. When a torque disturbance is present, the systems without the controller and with the EMPC controller are fully affected by the impact. In contrast, the system with the parallel structure controller minimizes the impact of the disturbance. This result shows that ANFIS estimates the output change caused by the impact and generates a control input in the opposite direction to compensate for the disturbance. In the error graphs between the reference input and the output, the system with the parallel structure controller of EMPC and ANFIS exhibits the smallest error. This result indicates that the parallel structure controller improves the system performance and stability by minimizing the effects of inertia and compensating for disturbances.

Figure 21 presents the quantitative performance metrics for the responses shown in Figure 20. The integral of absolute error (IAE) represents the total accumulated absolute error of the controller. The IAE indicate that the proposed EMPC-ANFIS controller reduces the error by 55% compared with the other controllers, thereby improving tracking accuracy. The integral of time-weighted absolute error (ITAE) represents the time-weighted absolute error. This result indicate that the EMPC-ANFIS controller notably decreases sensitivity to delayed errors and enhances long-term stability.

TABLE 4. Comparison between MPC and EMPC.

	MPC	EMPC
Total computation time	$50ms$	$31ms$
Average computation time per step	$17.7\mu s/step$	$11.0\mu s/step$
Memory footprint	$0.024MB$	$0.050MB$

The rise time (T_r) represents the time required to reach the reference value and is similar for all three controllers 0.11 s, which demonstrate that the proposed controller improves performance while maintaining response speed. The overshoot percentage (OS) represents the ratio exceeding the steady-state value. The EMPC-ANFIS controller exhibits an OS of 22.5%, which is 45% smaller than that achieved for EMPC (40.5%). These results demonstrate that the proposed parallel control structure effectively suppresses transient responses and achieves more stable control performance.

Table 4 compares MPC with EMPC under identical constraints in the experimental scenario shown in Figure 20. The average computation time is obtained by normalizing the total computation time by the number of calls in the upper loop. The actual number of steps in the upper loop is measured as 2824. The average computation time of EMPC is $11.0\mu s/step$, which is 1.6 times less than that of MPC ($17.7\mu s/step$). This result confirms that EMPC effectively reduces the computational burden under the same conditions. In addition, as the computation times of both controllers are $100\mu s$ for a sampling period of 0.1 s, they satisfy the real-time control requirement. The memory footprint of MPC (0.024 *MB*) is 2.1 times smaller than that of EMPC (0.050 *MB*) because EMPC stores the precomputed F, G, H, and K matrices for each region. However, the absolute size of 0.05 *MB* is sufficiently small to be negligible even in embedded systems.

Toub presents a control design methodology in which an ANFIS is trained by an MPC to minimize the electrical energy consumption of a building heating, ventilation, and air-conditioning (HVAC) system [51]. Although this approach overcomes the limitations of MPC associated with high computational requirements, controller performance is maintained only under limited operating conditions. Specifically, ANFIS can reproduce the performance of MPC only within the restricted scenarios used for training, while its performance degradation is observed in other environments. In contrast, the parallel control structure of EMPC and ANFIS proposed in this paper simultaneously improves the performance of a gimbal system and compensates for disturbances. EMPC reduces the real-time computational burden and consistently maintains improved system performance regardless of the training scenarios. ANFIS effectively preserves system performance by compensating for unexpected disturbances when they occur.

Table 5 presents a performance comparison between the MPC-trained ANFIS and the parallel structure of ANFIS and EMPC. Since the parallel structure controller is based on

TABLE 5. Performance comparison between MPC-Trained ANFIS and Parallel Structure of ANFIS–EMPC.

	MPC-trained ANFIS	Parallel structure
Computation time	slow	fast
Memory footprint	low	high
Generalization capability	limited	effective

TABLE 6. Computation time of the ANFIS.

	ANFIS
Total computation time	$0.704s$
Average computation time per step	$249\mu s/step$

EMPC, it achieves a shorter computation time than the MPC-trained ANFIS, while requiring a larger memory footprint due to the need to store critical regions computed offline. The MPC-trained ANFIS learns the control inputs generated by MPC and therefore reproduces MPC performance only under limited operating conditions, while its performance degrades outside the trained scenarios. In contrast, the parallel structure controller learns the effects of disturbances present in each training scenario, and thus effectively maintains system performance within the disturbance range covered by the training scenarios.

Table 6 presents the total and average computation times of the ANFIS controller. The total computation time during the entire simulation is 0.704 s, and the average computation time per step is measured as $249\mu s/step$. This corresponds to 0.25% of the sampling period, indicating that the controller has a sufficient timing margin for real-time operation. Therefore, the proposed parallel control structure enables real-time implementation in actual hardware.

The ANFIS compensation term is designed to preserve the feasibility and closed loop stability guaranteed by EMPC. ANFIS is introduced as an auxiliary controller that compensates for residual errors caused by disturbances and does not intervene in the EMPC optimization process or region partitioning structure. The final control input is generated by combining the EMPC output with the ANFIS compensation in a parallel structure, thereby maintaining the constraint handling and stability properties of EMPC. Moreover, the ANFIS output is structurally bounded due to the use of normalized membership functions and a finite rule base, and its input variables are restricted within predefined finite ranges. As a result, the combined control input remains bounded and does not violate the predefined input constraints, indicating that the ANFIS compensation does not undermine the feasibility of EMPC.

The methodological contribution of this study lies in a controller design approach that integrates EMPC and ANFIS in a parallel structure. EMPC serves as the primary controller that guaranties predictable real time control performance and feasibility while satisfying constraints based on precomputed control laws. ANFIS is introduced as an auxiliary controller to compensate for residual errors caused by disturbances, without modifying the EMPC optimization process or the region partitioning structure. This separation of roles enables the incorporation of learning based compensation while preserving the stability and constraint handling properties of model based control. The proposed parallel structure provides a practical and reproducible controller design framework that enhances robustness to disturbances while maintaining the model based guaranties of EMPC.

V. CONCLUSION

This study proposes a parallel control structure that combines EMPC and ANFIS to enhance the performance of the drone gimbal system. The EMPC is designed to ensure predictive optimization with low computational complexity. The ANFIS is implemented to improve adaptability and compensate modeling uncertainties and external disturbances. Simulations using the QUBE-Servo 3 platform model reveal that the proposed structure demonstrates superior control performance compared to conventional MPC and EMPC.

The performance improvement achieved by the parallel structure is attributed to the complementary roles of EMPC and ANFIS. EMPC determines the optimal control input based on precomputed regions, reducing the computational burden and ensuring real-time feasibility. At the same time, ANFIS adaptively compensates residual modeling errors and nonmodeled nonlinearities, effectively mitigating the influence of external disturbances such as voltage and torque variations. This cooperative mechanism allows the controller to maintain accurate and stable performance even under dynamic operating conditions, which highlights the practical potential of the proposed structure for real-time gimbal control applications.

In this study, the QUBE-Servo 3 is used to model the yaw-axis motion of the gimbal system. The use of QUBE-Servo 3 simplifies the gimbal system and primarily evaluates the performance of the proposed parallel structure of EMPC and ANFIS. Future research will involve experiments using a three-axis gimbal system incorporating roll, pitch, and yaw motions. Since the three-axis gimbal is the most commonly used configuration in practice, applying the proposed control method to such a system will allow for the evaluation of its performance in a more realistic and practical context. By proving the stability of the parallel structure controller for a three-axis gimbal system, we will demonstrate that the system operates within an appropriate range. The sampling period will also be adjusted to match that of practical gimbal systems. Additionally, future studies will involve experiments using an actual drone equipped with a gimbal system. Although the disturbances in this study were designed to reflect real environmental factors, they were ultimately implemented through computer simulations. While the simulated disturbances closely resemble real-world scenarios, the experiments were conducted in a virtual environment, which presents certain limitations. Therefore, future experiments will aim to introduce real-world disturbances,

enabling ANFIS to learn and compensate for the effects of actual environmental disturbances. In particular, experimental scenarios with wind-gust, broadband, and multi-axis disturbances will be designed to more accurately represent real environmental effects and more comprehensively evaluate controller robustness and adaptability. Furthermore, future studies will focus on introducing measurable signals as disturbances and developing a disturbance observer to estimate them. Reconstructing the ANFIS training scenarios based on real disturbance data, the system is able to maintain robust performance even under more complex disturbances. By training ANFIS using scenarios that include unseen disturbances, its robustness can be validated in complex real-world environments. In addition, future work will apply anti-windup and rate-limiting techniques to prevent integrator windup and excessive input variation and thus ensure actuator safety during real-hardware implementation. By addressing these aspects, future work will evaluate the performance and stability of the proposed control system under more realistic conditions.

REFERENCES

[1] H. Kang, H. Li, J. Zhang, X. Lu, and B. Benes, "FlyCam: Multitouch gesture controlled drone gimbal photography," *IEEE Robot. Autom. Lett.*, vol. 3, no. 4, pp. 3717–3724, Oct. 2018.

[2] J. F. Keane and S. S. Carr, "A brief history of early unmanned aircraft," *Johns Hopkins APL Tech. Dig.*, pp. 558–571, 2013.

[3] Ö. Villi and H. Yavuz, "The utilization of gimbal systems in unmanned aerial vehicles," *Adv. UAV*, vol. 4, no. 1, pp. 19–30, 2024.

[4] X. Wang, J. Zhao, X. Pei, T. Wang, T. Hou, and X. Yang, "Bioinspiration review of aquatic unmanned aerial vehicle (AquaUAV)," *Biomimetic Intell. Robot.*, vol. 4, no. 2, Jun. 2024, Art. no. 100154.

[5] A. Altan, "Performance of Metaheuristic optimization algorithms based on swarm intelligence in attitude and altitude control of unmanned aerial vehicle for path following," in *Proc. 4th Int. Symp. Multidisciplinary Stud. Innov. Technol. (ISMSIT)*, Oct. 2020, pp. 1–6.

[6] F. Santoso, M. A. Garratt, S. G. Anavatti, and I. Petersen, "Robust hybrid nonlinear control systems for the dynamics of a quadcopter drone," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 50, no. 8, pp. 3059–3071, Aug. 2020.

[7] C. Yanarateş and A. Altan, "Compact analysis of the necessity of padé approximation for delayed continuous-time models in LQR, H-infinity and root locus control strategies," *Black Sea J. Eng. Sci.*, vol. 7, no. 6, pp. 53–54, 2024.

[8] M. A. Al-Shareeda, M. A. Saare, and S. Manickam, "Unmanned aerial vehicle: A review and future directions," *Indonesian J. Electr. Eng. Comput. Sci.*, vol. 30, no. 2, pp. 778–786, May 2023.

[9] S. A. H. Mohsan, M. A. Khan, F. Noor, I. Ullah, and M. H. Alsharif, "Towards the unmanned aerial vehicles (UAVs): A comprehensive review," *Drones*, vol. 6, no. 6, p. 147, Jun. 2022.

[10] A. Altan and R. Hacıoğlu, "Model predictive control of three-axis gimbal system mounted on UAV for real-time target tracking under external disturbances," *Mech. Syst. Signal Process.*, vol. 138, Apr. 2020, Art. no. 106548.

[11] M. M. Abdo, A. R. Vali, A. R. Toloei, and M. R. Arvan, "Stabilization loop of a two axes gimbal system using self-tuning PID type fuzzy controller," *ISA Trans.*, vol. 53, no. 2, pp. 591–602, Mar. 2014.

[12] M. H. Ahmad, K. Osman, M. F. M. Zakeri, and S. I. Samsudin, "Mathematical modelling and PID controller design for two DOF gimbal system," in *Proc. IEEE 17th Int. Colloq. Signal Process. Its Appl. (CSPA)*, Mar. 2021, pp. 138–143.

[13] J. B. Rawlings, "Tutorial overview of model predictive control," *IEEE Control Syst. Mag.*, vol. 20, no. 3, pp. 38–52, Jun. 2000.

[14] J. Köhler, M. A. Müller, and F. Allgöwer, "Analysis and design of model predictive control frameworks for dynamic operation—An overview," *Annu. Rev. Control*, vol. 57, Jan. 2024, Art. no. 100929.

[15] M. L. Darby and M. Nikolaou, "MPC: Current practice and challenges," *Control Eng. Pract.*, vol. 20, no. 4, pp. 328–342, Apr. 2012.

[16] M. Diehl, H. G. Bock, J. P. Schlöder, R. Findeisen, Z. Nagy, and F. Allgöwer, "Real-time optimization and nonlinear model predictive control of processes governed by differential-algebraic equations," *J. Process Control*, vol. 12, no. 4, pp. 577–585, Jun. 2002.

[17] J. Drgoňa, J. Arroyo, I. C. Figueroa, D. Blum, K. Arendt, D. Kim, E. P. Ollé, J. Oravec, M. Wetter, D. Vrabie, and L. Helsen, "All you need to know about model predictive control for buildings," *Annu. Rev. Control*, vol. 50, pp. 190–232, Dec. 2020.

[18] C. E. Garcia, D. M. Prett, and M. Morari, "Model predictive control: Theory and practice—A survey," *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.

[19] M. Schwenzer, M. Ay, T. Bergs, and D. Abel, "Review on model predictive control: An engineering perspective," *Int. J. Adv. Manuf. Technol.*, vol. 117, nos. 5–6, pp. 1327–1349, 2021.

[20] A. Bemporad and M. Morari, "Robust model predictive control: A survey," in *Robustness in Identification and Control*. Cham, Switzerland: Springer, 2007, pp. 207–226.

[21] Y. Wang and S. Boyd, "Fast model predictive control using online optimization," *IEEE Trans. Control Syst. Technol.*, vol. 18, no. 2, pp. 267–278, Mar. 2010.

[22] S. J. Qin and T. A. Badgwell, "A survey of industrial model predictive control technology," *Control Eng. Pract.*, vol. 11, no. 7, pp. 733–764, Jul. 2003.

[23] A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos, "The explicit linear quadratic regulator for constrained systems," *Automatica*, vol. 38, no. 1, pp. 3–20, Jan. 2002.

[24] M. N. Zeilinger, C. N. Jones, and M. Morari, "Real-time suboptimal model predictive control using a combination of explicit MPC and online optimization," *IEEE Trans. Autom. Control*, vol. 56, no. 7, pp. 1524–1534, Jul. 2011.

[25] P. Tøndel, T. A. Johansen, and A. Bemporad, "An algorithm for multi-parametric quadratic programming and explicit MPC solutions," *Automatica*, vol. 39, no. 3, pp. 489–497, Mar. 2003.

[26] M. de la Pena, A. Bemporad, and C. Filippi, "Robust explicit MPC based on approximate multi-parametric convex programming," in *Proc. 43rd IEEE Conf. Decis. Control (CDC)*, Dec. 2004, pp. 2491–2496.

[27] I. Maurovic, M. Baotic, and I. Petrovic, "Explicit model predictive control for trajectory tracking with mobile robots," in *Proc. IEEE/ASME Int. Conf. Adv. Intell. Mechatronics (AIM)*, Jul. 2011, pp. 712–717.

[28] Z. Feng, J. Chen, W. Xiao, J. Sun, B. Xin, and G. Wang, "Learning hybrid policies for MPC with application to drone flight in unknown dynamic environments," 2024, *arXiv:2401.09705*.

[29] J.-S.-R. Jang, "ANFIS: Adaptive-network-based fuzzy inference system," *IEEE Trans. Syst., Man, Cybern.*, vol. 23, no. 3, pp. 665–685, Jun. 1993.

[30] J. R. Jang and C.-T. Sun, "Neuro-fuzzy modeling and control," *Proc. IEEE*, vol. 83, no. 3, pp. 378–406, 1995.

[31] J.-S.-R. Jang and C.-T. Sun, "Functional equivalence between radial basis function networks and fuzzy inference systems," *IEEE Trans. Neural Netw.*, vol. 4, no. 1, pp. 156–159, Jan. 1993.

[32] L. A. Zadeh, "Fuzzy sets," *Inf. Control*, vol. 8, no. 3, pp. 338–353, 1965.

[33] L. A. Zadeh, "Fuzzy logic," *Computer*, vol. 21, no. 4, pp. 83–93, 1988.

[34] C. C. Lee, "Fuzzy logic in control systems: Fuzzy logic controller. I," *IEEE Trans. Syst., Man, Cybern.*, vol. 20, no. 2, pp. 404–418, Apr. 1990.

[35] C. C. Lee, "Fuzzy logic in control systems: Fuzzy logic controller. II," *IEEE Trans. Syst., Man, Cybern.*, vol. 20, no. 2, pp. 419–435, Apr. 1990.

[36] B. Singh and A. K. Mishra, "Fuzzy logic control system and its applications," *Int. Res. J. Eng. Technol. (IRJET)*, vol. 2, no. 8, pp. 742–746, 2015.

[37] H.-X. Li and H. B. Gatland, "A new methodology for designing a fuzzy logic controller," *IEEE Trans. Syst., Man, Cybern.*, vol. 25, no. 3, pp. 505–512, Mar. 1995.

[38] W. Xie, S. K. Nguang, H. Zhu, Y. Zhang, and K. Shi, "A novel event-triggered asynchronous H_∞ control for T-S fuzzy Markov jump systems under hidden Markov switching topologies," *Fuzzy Sets Syst.*, vol. 443, pp. 258–282, Aug. 2022.

[39] S. Haykin, *Neural Networks: A Comprehensive Foundation*. Upper Saddle River, NJ, USA: Prentice-Hall, 1994.

[40] G. Ia, "Deep learning/ian goodfellow, Yoshua bengio and Aaron courville," *Adapt. Comput. Mach. Learn.*, 2016.

[41] C. M. Bishop, *Neural Networks for Pattern Recognition*. London, U.K.: Oxford Univ. Press, 1995.

[42] D. H. Nguyen and B. Widrow, "Neural networks for self-learning control systems," *IEEE Control Syst. Mag.*, vol. 10, no. 3, pp. 18–23, Apr. 1990.
[43] N. Walia, H. Singh, and A. Sharma, "Anfis: Adaptive neuro-fuzzy inference system—A survey," *Int. J. Comput. Appl.*, vol. 123, no. 13, pp. 32–38, 2015.
[44] N. Assanova, A. Khaimuldin, N. Khaimuldin, S. Alshynov, and T. Mukatayev, "Realisation of MPC algorithm for quanser qube-servo," *Sci. J. Astana IT Univ.*, pp. 42–56, Jun. 2023.
[45] S. Adarsh and S. S. Kumar, "Model identification and position control of quanser qube servo DC motor," in *Proc. Int. Conf. Adv. Electr. Comput. Technol.*, 2021, pp. 1321–1335.
[46] M. Morari and N. L. Ricker, *Model Predictive Control Toolbox User's Guide*. Natick, MA, USA: MathWorks, 1998.
[47] M. Guo, Y. Hao, G. Chen, K. Y. Lee, and L. Sun, "Energy balance error compensated extended-state Kalman filter-based model predictive control of a 300 mw boiler-turbine unit," *Chem. Eng. Res. Design*, vol. 217, pp. 328–341, May 2025.
[48] M. Guo, Y. Hao, K. Y. Lee, and L. Sun, "Extended-state Kalman filter-based model predictive control and energy-saving performance analysis of a coal-fired power plant," *Energy*, vol. 314, Jan. 2025, Art. no. 134169.
[49] S. Ünsal and I. Aliskan, "Performance analysis of fuzzy logic controllers having Mamdani and Takagi–Sugeno inference methods by using unique software and toolbox," in *Proc. Nat. Conf. Electr., Electron. Biomed. Eng. (ELECO)*, Dec. 2016, pp. 237–241.
[50] C. Loganathan and K. Girija, "Hybrid learning for adaptive neuro fuzzy inference system," *Int. J. Eng. Sci.*, vol. 2, no. 11, pp. 6–13, 2013.
[51] M. Toub, M. Shahbakhti, R. D. Robinett, and G. Aniba, "MPC-trained ANFIS for control of MicroCSP integrated into a building HVAC system," in *Proc. Amer. Control Conf. (ACC)*, Jul. 2019, pp. 241–246.

KANG-HYEON CHOI is currently pursuing the B.S. degree in electrical engineering with Kookmin University, Republic of Korea. His research interests include vehicle control and robotics.

HYUK-JUN CHANG (Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Seoul National University, South Korea, in 1998 and 2004, respectively, and the Ph.D. degree in electrical and electronic engineering from Imperial College London, U.K., in 2009. He is currently a Professor with the School of Electrical Engineering, Kookmin University, Republic of Korea. His research interests include nonlinear control theory and nonlinear systems analysis.